



Cómo integrarr distintas IAs desde Python

Aprende a integrar OpenAI, Claude, Gemini y Ollama en tus proyectos Python usando sus APIs REST.

Introducción

APIs de IA en Python



Desde Python puedes enviar preguntas a **múltiples IAs** como OpenAI, Claude, Gemini u Ollama, independientemente del lenguaje en que estén implementadas internamente

Introducción

APIs de IA en Python

Todas comparten un patrón común: exponen una **API HTTP/REST** que puedes consumir fácilmente desde Python.

01

API Key

Obtén tu clave de acceso del proveedor

03

Formato JSON

Conoce la estructura de petición y respuesta

02

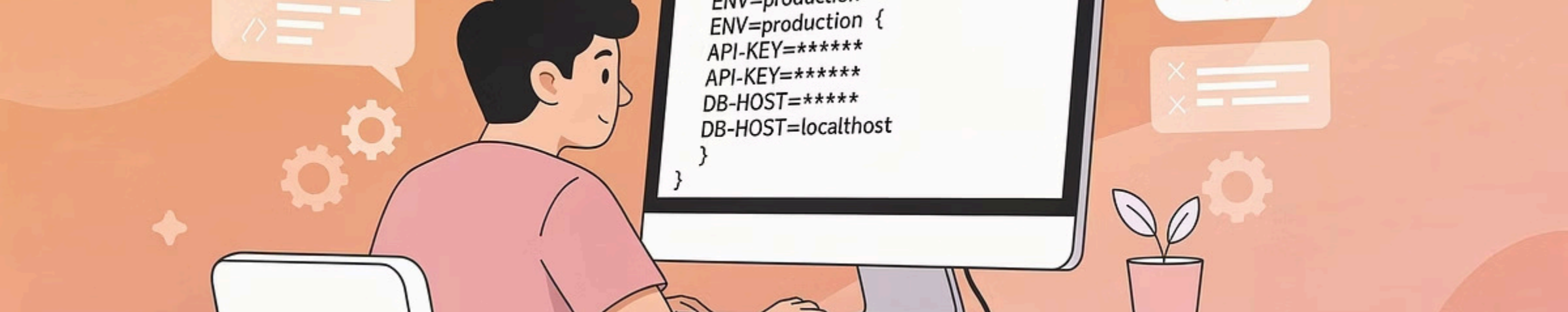
URL Endpoint

Identifica la dirección de la API

04

Librería Python

Usa requests o la SDK oficial



Configuración inicial

Patrón genérico para integrar cualquier IA



Requisitos básicos

- Python 3.x instalado
- Librería `requests`
- API Key del proveedor



Variables de entorno

Guarda tus claves de forma segura usando archivos `.env` con `python-dotenv`

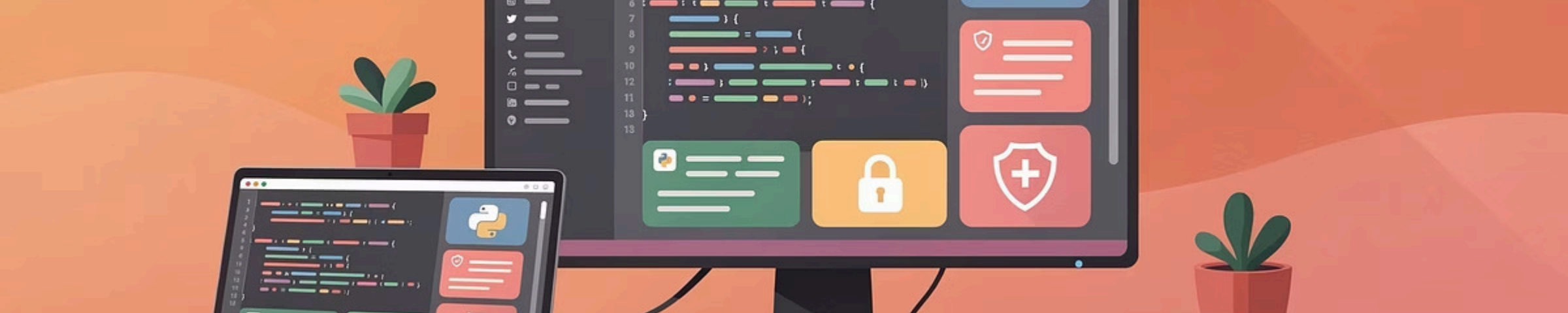


Función genérica

Crea una función reutilizable que maneje peticiones HTTP con autenticación



Buena práctica: Añade `.env` a tu `.gitignore` para no subir tus claves a repositorios públicos.



Gestión segura de claves

Configurar variables de entorno con .env

Instalación y configuración

```
pip install python-dotenv
```

Crea un archivo `.env` en la raíz de tu proyecto:

```
# Claves de IAs  
OPENAI_API_KEY=tu_key_aqui  
ANTHROPIC_API_KEY=tu_key_aqui  
GEMINI_API_KEY=tu_key_aqui
```

Uso en Python

```
import os  
from dotenv import load_dotenv  
  
load_dotenv()  
  
openai_key = os.getenv("OPENAI_API_KEY")  
claude_key = os.getenv("ANTHROPIC_API_KEY")
```



Archivo `.env`

Define tus variables



`load_dotenv()`

Carga al entorno



`os.getenv()`

Lee valores

Integración con proveedores principales

OpenAI (ChatGPT)

Accede a GPT-4.1, GPT-4o y otros modelos. Instalación:

```
pip install openai
```

Anthropic (Claude)

Modelos Claude 3.5 Sonnet y variantes. Instalación:

```
pip install anthropic
```

Google Gemini

Gemini 1.5 Flash y Pro. Instalación:

```
pip install google-generativeai
```

Nota: Servicio de pago, exceptuando Google Gemini que tiene un free tier

Costes en la API de OpenAI

Aspectos clave del pricing

- No hay plan gratuito para la **API** (sí hay modelos gratuitos en la interfaz web).
- Coste basado en **tokens**:
 - Tokens de entrada (input).
 - Tokens de salida (output).
 - Tokens en caché (cached), más baratos.

Factores que aumentan el coste

- Conversaciones largas con mucho historial.
- Prompts y respuestas muy extensos.
- Uso de modelos más caros.
- Gran número de llamadas a la API.

 **Recomendación:** Para pruebas y aprendizaje: usar gpt-5-nano y/o modelos locales con Ollama. Monitorizar uso en el panel de OpenAI.

Conclusiones

Todas las IAs siguen el mismo flujo:

1. API key,
2. endpoint,
3. formato JSON
4. Usar la librería.

Usa siempre variables de entorno para las claves. Nunca las incluyas directamente en el código ni las subas a repositorios públicos.

OpenAI, Claude, Gemini y modelos locales te dan flexibilidad para elegir según tus necesidades de coste, privacidad y funcionalidades.

